laborch: Package Manual

Version 0.1.0 (Phase 0)

LabOrch project

October 2026

Contents

1. Concepts and vocabulary 1
2. Installation 1
3. Quick start 2
4. Package layout 2
5. Module reference 3
6. Recipes 9
7. Data conventions 9
8. Testing 10
9. Known limitations and roadmap 10

`laborch` is the Python core of LabOrch, a computer-assisted orchestration tool. Given a **Target** sound and an **Ensemble** of instruments, it finds the combination of recorded notes whose summed spectrum best matches the Target. The package gathers the logic of the 2019–2020 notebooks (1.5, 1.7, 1.11, 1.12 and the `tool*.py` widget scripts) into tested, importable modules.

1. Concepts and vocabulary

Term	Meaning	In code
Bank	Every recorded note of every instrument (1,515 sounds, 33 instruments), each stored as an averaged spectrum	`Bank`
Ensemble	The instruments the composer writes for, e.g. a string quartet	a list of instrument codes, `["vln", "vla", "vlc"]`
Palette	Every Bank sound the Ensemble can play (notes, modes, dynamics), as a matrix of spectra	`Palette` (from `Bank.select`)
Target	The sound to imitate: a Bank sound or an uploaded recording	a 2,048-element NumPy array
Recommendation	The weighted set of Palette sounds that best reproduces the Target	`Fit`

The model (Rose & Hetrick 2009, “RH2009”) treats the Target spectrum T as a weighted sum of Palette spectra X_j:

$$T(f_i) \;\approx\; \sum_j X_{ij}\,\alpha_j, \qquad \alpha_j \geq 0$$

The weights α are found by non-negative least squares (NNLS), optionally weighted per frequency bin by $1/\sigma_i$ as in *Numerical Recipes* §15.4.

2. Installation

Requirements: Python $\geq$ 3.9, NumPy, SciPy. Audio features (reading recordings, splitting, mixing) also need **pydub** and **ffmpeg** on the `PATH`.

From the `LabOrch` folder:

```
pip install -e ".[dev]"     # package + pydub + pytest, editable
# or, without installing:
#   import sys; sys.path.insert(0, "path/to/LabOrch")
```

On Windows, ffmpeg can be installed with `winget install ffmpeg`.

The package finds the data relative to its own location (the repository root). To use data elsewhere, set the environment variable `LABORCH_ROOT` before importing (see 5.1).

3. Quick start

```
from laborch import Bank, nnls_fit

bank = Bank.load()                      # first call ~1 min, then cached
target_name = "trb-f2-ord-mf"           # trombone F2, mf
target = bank.spectrum(target_name)

palette = bank.select(instruments=["clb", "bcl"],   # Ensemble: clarinets
                      exclude=[target_name])
fit = nnls_fit(palette, target)

print(fit.summary(5))
# nnls: 62 sounds, relative error 0.529
#     0.1415  bcl-f2-ord-mf
#     0.1171  bcl-f3-ord-mf
#     0.0729  clb-f5-ord-mf
#     ...

best3 = fit.top(3, refit=True)          # keep 3 sounds, re-solve on them
best3.ranked()                          # [('bcl-f2-ord-mf', ...), ...]

from laborch.audio import render_fit   # needs pydub + ffmpeg
render_fit(bank, best3, n=3, path="trb_from_clarinets.wav")
```

The notebook `Notebooks/2.0_laborch_quickstart.ipynb` walks through the same example with plots and playback.

4. Package layout

```
LabOrch/
├── laborch/
│   ├── __init__.py   public API (re-exports)
│   ├── config.py     paths and FFT constants
│   ├── bank.py       sound names, Bank, Palette
│   ├── spectra.py    audio loading, averaged-FFT pipeline, splitting
│   ├── solve.py      DOT, pseudo-inverse and NNLS decompositions; Fit
```

```
│   ├── audio.py      gains and mixing for playback
│   └── synth.py      synthetic tones, spectrum-to-audio resynthesis
├── tests/            pytest suite (28 tests)
├── docs/             this manual
└── pyproject.toml
```

Names importable directly from `laborch`:

`Bank`, `Palette`, `SoundName`, `parse_name`, `INSTRUMENT_NAMES`, `Fit`, `dot`, `nnls_fit`, `pinv_fit`, `chi2`, `make_fft`, `spectral_peaks`, `tone`, `spectrum_to_audio`.

Audio helpers are imported from their module (`laborch.audio`, `laborch.spectra`) so that the core works without pydub.

---

5. Module reference

5.1 `laborch.config`

Constants only; no functions. Change paths here, not in calling code.

Name	Value	Meaning
`ROOT`	repository folder, or `$LABORCH_ROOT`	base for every path below
`BANK_DIR`	`ROOT/Soundbank_McGill`	McGill Bank folder
`FFT_DIR`	`BANK_DIR/FFTBank`	current spectra, one `.txt` per sound
`SOUND_DIRS`	`Soundbank_WB`, `Soundbank_Str`, `Soundbank_PianoPercSax`	the `.aif` recordings
`OLD_FFT_DIR`	`ROOT/OldFFTBank`	2009 `.nbb` spectra (reference)
`MISC_SOUNDS_DIR`	`ROOT/MiscSounds`	extra Targets (trumpet mutes, multiphonics)
`CACHE_FILE`	`ROOT/.cache/fftbank.npz`	binary cache of the Bank
`SAMPLE_RATE`	44100	Hz
`NFFT`	4096	Hamming window length (samples)
`HOP`	512	step between windows (samples)
`NBINS`	2048	bins kept per spectrum
`BIN_HZ`	10.766...	Hz per bin (`SAMPLE_RATE / NFFT`)
`SHAVE`	0.15	fraction trimmed from each end of a note

`LABORCH_ROOT` is read once, at import time.

5.2 `laborch.bank`

Constants **`INSTRUMENT_NAMES`** — `dict[str, str]`, the 33 instrument codes of the current Bank mapped to names, e.g. `"vln"` → `"Violin"`, `"clb"` → `"Bb Clarinet"`, `"tpc"` → `"C Trumpet"`.

`MODE_NAMES` — `dict[str, str]`, performance-mode codes: `ord` ordinary, `mtd` muted, `nhr` natural harmonic, `ahr` artificial harmonic, `hmo` harmon mute, `stm` straight mute, `ped` pedal tone, `bwd` bowed, `hdm`

hard mallet, `bbt` brass beater. Meanings are inferred from the codes and should be confirmed against the MUMS index PDFs.

`parse_name(name) -> SoundName` Parses a Bank sound name, with or without extension.

```
>>> s = parse_name("ctb-2fs4-nhr-mf.txt")
>>> s.instrument, s.string, s.pitch, s.mode, s.dynamic
('ctb', 2, 'fs4', 'nhr', 'mf')
```

Raises `ValueError` for names that do not follow the scheme.

`class SoundName` (frozen dataclass)

Field	Type	Example
`instrument`	`str`	`"vln"`
`note`	`str`	`"a"`, `"cs"` (sharps as `s`)
`octave`	`int`	`4`
`mode`	`str`	`"ord"`
`dynamic`	`str`	`"pp"`, `"mf"`, `"ff"`
`string`	`int` or `None`	`2` in `ctb-2fs4-nhr-mf`
`variant`	`str`	`"o"` / `"x"` in `vln-a4o-ord-mf` (probably open / stopped string)

Property	Returns
`pitch`	`"a4"`
`midi`	MIDI number, C4 = 60, A4 = 69
`freq`	equal-tempered fundamental in Hz, A4 = 440
`str(s)`	the original name

`class Palette` (dataclass)

Attribute	Type	Meaning
`names`	`list[str]`	sound names, one per column
`matrix`	`ndarray (2048, n)`	spectra as **columns** (the basis X of RH2009)

`len(palette)` is the number of sounds. Palettes are normally made with `Bank.select`, but any set of spectra can be wrapped: `Palette(["a", "b"], np.column_stack([spec_a, spec_b]))`.

`class Bank` All Bank spectra, one **row** per sound, rows sorted by name.

Constructor `Bank(names, spectra, source="")` — `spectra` must have shape `(len(names), 2048)`. Every name must parse with `parse_name`.

Class methods

Method	Description
`Bank.load(fft_dir=FFT_DIR, use_cache=True, cache_file=CACHE_FILE)`	Loads the current Bank from `FFTBank/*.txt`. The first call reads 1,515 text files (about a minute on Dropbox) and writes `.cache/fftbank.npz`; later calls load the cache in about a second. The cache is used only if it holds exactly the same set of names; pass `use_cache=False` to force a rebuild.
`Bank.load_old(nbb_dir=OLD_FFT_DIR)`	Loads the 2009 `.nbb` spectra (2,048 float32 values each). One malformed name (`obo-f5-c6.nbb`) is skipped with a warning. Old instrument codes differ (`cla`, `hrn`, `tpt`, `mar`, `btr`).

Attributes

Attribute	Meaning
`names`	sorted list of sound names
`spectra`	`ndarray (n_sounds, 2048)`
`info`	`dict[name, SoundName]`
`source`	folder the Bank was read from
`instruments`	sorted list of instrument codes present (property)

Methods

Method	Returns	Description
`len(bank)`	`int`	number of sounds
`name in bank`	`bool`	membership
`spectrum(name)`	`ndarray (2048,)`	one sound's spectrum (a view; copy before modifying)
`select(instruments=None, modes=None, dynamics=None, midi_range=None, exclude=())`	`Palette`	builds a Palette. Each filter is optional; `midi_range=(lo, hi)` is inclusive; `exclude` removes named sounds, e.g. the Target. Raises `ValueError` if nothing matches.
`sound_path(name)`	`Path`	the `.aif` recording for a sound; `FileNotFoundError` if absent

```
quartet = bank.select(instruments=["vln", "vla", "vlc"],
                      modes=["ord", "mtd"],
                      midi_range=(48, 84))
```

`frequencies() -> ndarray` Centre frequency in Hz of each of the 2,048 bins (bin 0 = DC). Use it as the x-axis when plotting spectra.

5.3 `laborch.spectra`

The averaged-spectrum pipeline used to build the Bank, plus audio I/O. Functions marked † need pydub and ffmpeg.

`make_fft(samples, shave=0.15, normalize=True) -> ndarray` The Bank's spectrum recipe (notebook 1.11):

1. drop the first and last `shave` fraction of the samples (attack and decay);
2. slide a 4,096-sample Hamming window in steps of 512;
3. combine the frames as $\left(\sum_{\text{frames}} |X_k|^2\right)^{1/2}/n$, keeping bins 0–2047;
4. if `normalize`, divide by the sum (unit area), as every Bank file is.

`samples` is a 1-D array of mono samples at 44.1 kHz. Raises `ValueError` if the sound is shorter than one window after trimming. Use `normalize=False` to keep the absolute level.

`fft_of_file(path, **kw) -> ndarray` † `make_fft(load_samples(path), **kw)`. Recomputing a Bank file this way matches the stored spectrum to about 0.1% of its peak.

`load_segment(path) -> pydub.AudioSegment` † Reads any ffmpeg-readable file (AIF, WAV, MP3, ...).

`segment_samples(segment) -> ndarray` † Mono samples of an `AudioSegment` as floats in the file's integer scale (the scale used to build the Bank). Warns if the sample rate is not 44.1 kHz.

`load_samples(path) -> ndarray` † `segment_samples(load_segment(path))`.

`split_on_silence(path, min_silence_len=500, silence_thresh=-50, keep_silence=100) -> list` † Splits a multi-note recording into one `AudioSegment` per note (notebooks 1.8, 1.9, 1.12). Times in ms, threshold in dBFS.

```
from laborch.spectra import split_on_silence, segment_samples, make_fft
notes = split_on_silence("MiscSounds/Trumpet-mutes_Basson-multiphics/11 Track 11.aiff")
cup_c5 = make_fft(segment_samples(notes[2]))
```

`spectral_peaks(spec, rel_height=0.02, max_hz=None) -> list` Local maxima of a spectrum as `[(Hz, height relative to the tallest), ...]`, lowest frequency first. Peaks below `rel_height` × the tallest, or above `max_hz`, are dropped.

```
>>> spectral_peaks(make_fft(tone(130.81, [1, 4, 10], [1, .5, .25])), max_hz=3000)
[(129.2, 1.0), (527.6, 0.45), (1302.8, 0.21)]
```

5.4 `laborch.solve`

`nnls_fit(palette, target, sigma=None, maxiter=None) -> Fit` Non-negative least squares (Lawson–Hanson, via `scipy.optimize.nnls`):

$$\min_{\alpha} \sum_i \left(\frac{T_i - \sum_j X_{ij}\,\alpha_j}{\sigma_i} \right)^2 \quad \text{subject to} \quad \alpha_j \geq 0$$

`sigma` is an optional array of 2,048 positive per-bin uncertainties; `None` weights every bin equally (the 2020 behaviour). This is the main solver.

`pinv_fit(palette, target, sigma=None) -> Fit` Unconstrained least squares via the pseudo-inverse — RH2009's "SVD" method. Weights can be negative (an instrument "subtracting" sound), so use it only for comparison.

`dot(palette, target, normalize=False) -> Fit` RH2009's "DOT" method: the projection $p_j = \sum_i T_i X_{ij}$ of the Target on each sound alone. With `normalize=True` it is cosine similarity (shape only). The weights are scores, not mixing amounts; use DOT to rank or prefilter, not to build a mix.

`chi2(target, model, sigma=None) -> float` Sum of squared, optionally σ-weighted, residuals.

`class Fit` (dataclass) The result of any solver: a Recommendation before playability rules.

Attribute	Meaning
`names`	Palette sound names
`weights`	`ndarray`, one weight per name (same order)
`target`	the Target spectrum
`matrix`	the Palette matrix used
`method`	`"nnls"`, `"pinv"`, `"dot"`, `"dot-cos"`, or `"nnls/top3"` etc.
`sigma`	the σ used, or `None`

Property	Returns
`model`	fitted spectrum `matrix @ weights`
`chi2`	χ^2 of the fit (σ-weighted if σ was given)
`relative_error`	$\|T - \text{model}\|/\|T\|$, unweighted; 0 is perfect
`order`	indices sorted by weight, largest first

Method	Returns	Description
`ranked(n=None, nonzero=True, rtol=0.0)`	`list[(name, weight)]`	largest first; with `nonzero`, drops weights $\leq$ `rtol` $\times$ `max` (use `rtol=1e-9` to hide round-off)
`top(n, refit=False)`	`Fit`	keeps the n largest weights. `refit=False` keeps their values (as the 2020 mock-up did); `refit=True` re-solves NNLS on those n sounds, which never increases χ^2
`kept_fraction(n)`	`float`	share of total weight in the top n
`summary(n=10)`	`str`	readable report

5.5 `laborch.audio`

Needs pydub and ffmpeg (except `weights_to_db`).

`weights_to_db(weights, floor_db=-60.0) -> ndarray` Gain of each weight relative to the largest: $20 \log_{10}(w/w_{\max})$. Zero weights get `floor_db`. Approximate, because Bank spectra are normalized and do not carry the recordings' own loudness.

`mix(bank, names, gains_db) -> AudioSegment` Overlays the named Bank recordings, each shifted by its gain in dB. The result is as long as the longest sound.

`render_fit(bank, fit, n=8, path=None) -> AudioSegment` Mixes the top-n sounds of a `Fit` at their relative gains; writes a file if `path` is given (format from its extension, e.g. `.wav`, `.mp3`). In Jupyter, the returned segment plays inline.

5.6 `laborch.synth`

Synthetic sounds (notebooks 1.2 and 2.1). Pure NumPy: no pydub needed.

`tone(f0, partials=(1,), amps=None, seconds=1.0, sr=44100, decay=0.0, phases=None, peak=1.0) -> ndarray` A sum of sine partials at multiples of `f0`:

$$x(t) = e^{-\lambda t} \sum_k a_k \sin(2\pi\, p_k f_0\, t + \phi_k)\,, \qquad \lambda = \texttt{decay}$$

Parameter	Meaning
`f0`	fundamental in Hz; `parse_name("mrb-c3-ord-mf").freq` gives 130.81
`partials`	multiples of `f0`; need not be integers (e.g. 3.9 for a real bar)
`amps`	amplitude of each partial (default all 1)
`seconds`, `sr`	duration and sample rate
`decay`	exponential decay rate in 1/s; 0 = steady
`phases`	starting phase of each partial in radians (default 0)
`peak`	largest absolute sample after scaling; `2**15 - 1` gives 16-bit scale

Raises `ValueError` if the lists differ in length or a partial reaches the Nyquist frequency (22,050 Hz).

`spectrum_to_audio(spec, seconds=1.0, sr=44100, seed=0, threshold=1e-4, peak=1.0) -> ndarray` Additive resynthesis of a magnitude spectrum. Every bin k above `threshold` $\times$ the maximum becomes a sine at $k \times 10.77$ Hz with amplitude `spec[k]` and a random phase (`seed` makes it repeatable). Works on Bank spectra, `Fit.model` and `make_fft` output.

Because the Bank stores magnitudes only, the phases are invented: the result keeps the timbre (which partials, how strong) but not the original waveform, attack or decay. A round trip spectrum $\rightarrow$ audio $\rightarrow$ spectrum keeps the shape with a cosine similarity of about 0.97.

```
from laborch.synth import tone, spectrum_to_audio
from IPython.display import Audio
x = tone(130.81, partials=[1, 4, 10], amps=[1, 0.5, 0.25])
Audio(spectrum_to_audio(make_fft(x)), rate=44100)
```

6. Recipes

A Target from a recording

```
from laborch.spectra import fft_of_file
target = fft_of_file("my_target.wav")         # 44.1 kHz, steady 1–2 s
fit = nnls_fit(bank.select(instruments=["flu", "clb", "obo"]), target)
```

Compare the RH2009 methods

```
pal = bank.select(instruments=["flu"])
t = bank.spectrum("vln-a4x-ord-mf")
for f in (dot(pal, t, normalize=True), pinv_fit(pal, t), nnls_fit(pal, t)):
    print(f.method, f.ranked(3, nonzero=False))
```

A synthetic Target (notebook 2.1)

```
from laborch.synth import tone
f0 = parse_name("mrb-c3-ord-mf").freq
spec = make_fft(tone(f0, partials=[1, 4, 10], amps=[1, 0.5, 0.25]))
print(nnls_fit(bank.select(instruments=["mrb"]), spec).summary(3))
# mrb-c3 0.657, mrb-c5 0.298, mrb-e6 0.148: a marimba C3 doubled
# on its 4th (C5) and 10th (about E6) partials
```

Try a frequency weighting (relative error per bin)

```
import numpy as np
sigma = np.maximum(target, 1e-6)              # sigma_i proportional to T_i
fit_rel = nnls_fit(palette, target, sigma=sigma)
```

Plot Target against a Recommendation

```
import matplotlib.pyplot as plt
from laborch.bank import frequencies
f = frequencies(); r = fit.top(4, refit=True)
plt.semilogy(f, target, label="Target"); plt.semilogy(f, r.model, label="Top 4")
plt.xlim(0, 5000); plt.xlabel("Hz"); plt.legend()
```

One note per instrument (manual, until Phase 2) — keep the best weight per instrument and refit:

```
best = {}
for name, w in fit.ranked():
    best.setdefault(bank.info[name].instrument, name)
import numpy as np
from laborch import Palette
pal1 = Palette(list(best.values()),
               np.column_stack([bank.spectrum(n) for n in best.values()]))
print(nnls_fit(pal1, target).summary())
```

7. Data conventions

- **Sound names** — `inst-pitch-mode-dyn`, e.g. `vln-a4x-ord-mf`. Sharps as `s` (`cs4`); optional string number before the pitch (`ctb-1b4-nhr-mf`); optional one-letter variant after it (`a4o`, `a4x`).
- **Spectra** — 2,048 bins, bin k centred at $k \times 10.77$ Hz (bin 0 = DC, last bin $\approx$ 22 kHz); magnitude, not power; normalized to unit sum.

- **Bank coverage** — 1,515 sounds; nearly all `mf`, with `pp` and `ff` only for piano.
- **Units of weights** — because spectra are unit-sum, a weight is the sound's share of the fitted spectrum, not a playing level.

8. Testing

```
cd LabOrch
pytest            # 28 tests, about 15 s once the Bank cache exists
pytest -v -k ravel
```

Test file	Checks
`test_names.py`	name parsing, MIDI numbers, every Bank name parses, unit normalization
`test_solve.py`	NNLS recovers an exact synthetic mixture, σ weighting, `top`/refit, dB gains
`test_spectra.py`	`make_fft` rebuilds four stored Bank spectra from their `.aif` files; pure-tone peak
`test_regression_notebook_1_5.py`	violin A4 from flutes matches notebook 1.5 to three decimals; pseudo-inverse goes negative, NNLS does not
`test_synth.py`	`tone` partials, amplitudes and decay; the synthetic C3 decomposes into marimba C3 + C5 + E6; spectrum-to-audio round trip
`test_regression_ravel.py`	RH2009 *Daphnis et Chloé* example: the woodwind chord is closer to the cup mute than the straight mute

Audio tests are skipped automatically if pydub or ffmpeg is missing; data tests are skipped if the Bank folders are absent.

9. Known limitations and roadmap

Limitation	Planned in
Spectra are unit-normalized magnitudes; loudness is lost and mixing is only approximately linear	Phase 1 (power spectra, loudness model)
Fixed 15% trim instead of detecting the sustain	Phase 1
Every frequency bin counts equally ($\sigma = 1$); plain χ^2 nearly ties the two mutes in the Ravel test while shape measures prefer the cup mute	Phase 2 (choice of σ)
No playability: several notes may be assigned to one player	Phase 2 (one-note-per-player search)
No web interface, score export, morphing or creative mode	Phases 3–5

See `Reading/LabOrch_Review_and_Plan.docx` for the full plan.